

P3652R0

Constexpr floating-point <charconv> functions

Published Proposal, 2025-03-15

Author:

[Lénárd Szolnoki](#)

Audience:

SG18, LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

- 1 Proposal**
- 2 Motivation**
- 3 Implementation experience**
- 4 Wording**
 - 4.1 [charconv.syn]
 - 4.2 [charconv.to.chars]
 - 4.3 [charconv.from.chars]
 - 4.4 [version.syn]
- 5 Acknowledgements**
- References**
 - Informative References

§ 1. Proposal

I propose to make the `to_chars` and `from_chars` functions for **floating-point types** usable in constant expressions.

§ 2. Motivation

[P2291R3] added `constexpr` to the integer overloads of `from_chars` and `to_chars`. The floating-point overloads were left out of consideration, as at the time there was little implementation experience even without `constexpr`. Quoting [P2291R3] (1.2.2 Floating point):

`std::from_chars/std::to_chars` are probably the most difficult to implement parts of a standard library. As of January 2021, only one of the three major implementations has full support of [P0067R5]:

Since then, standard libraries implemented the floating-point overloads of `from_chars` and `to_chars`. Popular 3rd party implementations of these functions also made their implementations `constexpr`.

In addition to the original motivation in [P2291R3], the utility of having all of the `<charconv>` functions available during constant evaluation is also greatly increased in combination with [P1967R14] (`#embed`), [P2741R3] (user-generated `static_assert` messages) and [P2758R4] (emitting messages at compile time).

§ 3. Implementation experience

I contributed to the [fast_float] and [dragonbox] projects to make their implementation of `from_chars` and `to_chars` usable in constant expressions.

[fast_float] has an implementation of `from_chars` limited to decimal string representation of floating-point numbers, with state-of-the-art runtime performance. Making this implementation usable from constant expressions was relatively straightforward. The bulk of the work was adding `constexpr` to all of the function declarations, with some other minor changes here and there, using features available since C++20:

- `std::is_constant_evaluated()` made it possible to navigate around compiler built-ins and other `constexpr`-hostile constructs (intrinsics, getting current rounding mode) and use the already available generic fallback implementation instead.

- `std::bit_cast` replaced `memcpy` for type punning.
- `std::copy` (`constexpr` since C++20) and related algorithms replaced `memcpy` for copying trivial ranges.

[dragonbox] has an implementation of `to_chars` limited to decimal string representation of floating-point numbers, with state-of-the-art runtime performance. Making it work in constant expressions was similarly straightforward.

I am not aware of any implementation for the hexadecimal floating-point parts of these functions. I expect little implementation difficulty of handling hexadecimal floating-point representations, given that the algorithms for handling the decimal representation are vastly more complicated.

§ 4. Wording

Wording is relative to [N5001].

§ 4.1. [charconv.syn]

```
// 28.2.2, primitive numerical output conversion
struct to_chars_result { // freestanding
    char* ptr;
    errc ec;
    friend bool operator==(const to_chars_result&, const to_chars_result&) = d
    constexpr explicit operator bool() const noexcept { return ec == errc{}; }
};
```

```
constexpr to_chars_result to_chars(char* first, char* last, // freestanding
                                   integer-type value, int base = 10);
to_chars_result to_chars(char* first, char* last, // freestanding
                        bool value, int base = 10) = delete;
```

```
constexpr to_chars_result to_chars(char* first, char* last, // freestanding-
                                   floating-point-type value);
constexpr to_chars_result to_chars(char* first, char* last, // freestanding-
                                   floating-point-type value, chars_format f
constexpr to_chars_result to_chars(char* first, char* last, // freestanding-
                                   floating-point-type value, chars_format f
```

```
// 28.2.3, primitive numerical input conversion
struct from_chars_result { // freestanding
```

```

const char* ptr;
errc ec;
friend bool operator==(const from_chars_result&, const from_chars_result&)
constexpr explicit operator bool() const noexcept { return ec == errc{}; }
};

constexpr from_chars_result from_chars(const char* first, const char* last,
                                       integer-type& value, int base = 10);

constexpr from_chars_result from_chars(const char* first, const char* last,
                                       floating-point-type& value,
                                       chars_format fmt = chars_format::general
}

```

§ 4.2. [charconv.to.chars]

```

constexpr to_chars_result to_chars(char* first, char* last, floating-point-t
<
...
constexpr to_chars_result to_chars(char* first, char* last, floating-point-t
<
...
constexpr to_chars_result to_chars(char* first, char* last, floating-point-t
<

```

§ 4.3. [charconv.from.chars]

```

constexpr from_chars_result from_chars(const char* first, const char* last,
<
<

```

§ 4.4. [version.syn]

```

#define __cpp_lib_constexpr_charconv 202207L DATE-OF-ADOPTION // freestanding
<

```

§ 5. Acknowledgements

I would like to thank Daniel Lemire and the contributors of [\[fast_float\]](#), as well as Junekey Jeon and the contributors of [\[dragonbox\]](#) for their work on their open-source libraries. Their efforts have been invaluable for making this proposal possible.

§ References

§ Informative References

[DRAGONBOX]

Junekey Jeon. *Dragonbox*. URL: <https://github.com/jk-jeon/dragonbox>

[FAST_FLOAT]

Daniel Lemire. *fast_float number parsing library*. URL: https://github.com/fastfloat/fast_float

[N5001]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/n5001.pdf>

[P0067R5]

Jens Maurer. *Elementary string conversions*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0067r5.html>

[P1967R14]

JeanHeyd Meneide; Shepherd (Shepherd's Oasis LLC). *#embed - a scannable, tooling-friendly binary resource inclusion mechanism*. URL: <https://isocpp.org/files/papers/P1967R14.html>

[P2291R3]

Daniil Goncharov; Alexander Karaev. *Add Constexpr Modifiers to Functions to `_chars` and from `_chars` for Integral Types in `<charconv>` Header*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2291r3.pdf>

[P2741R3]

Corentin Jabot. *user-generated static `_assert` messages*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2741r3.pdf>

[P2758R4]

Barry Revzin. *Emitting messages at compile time*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2758r4.html>